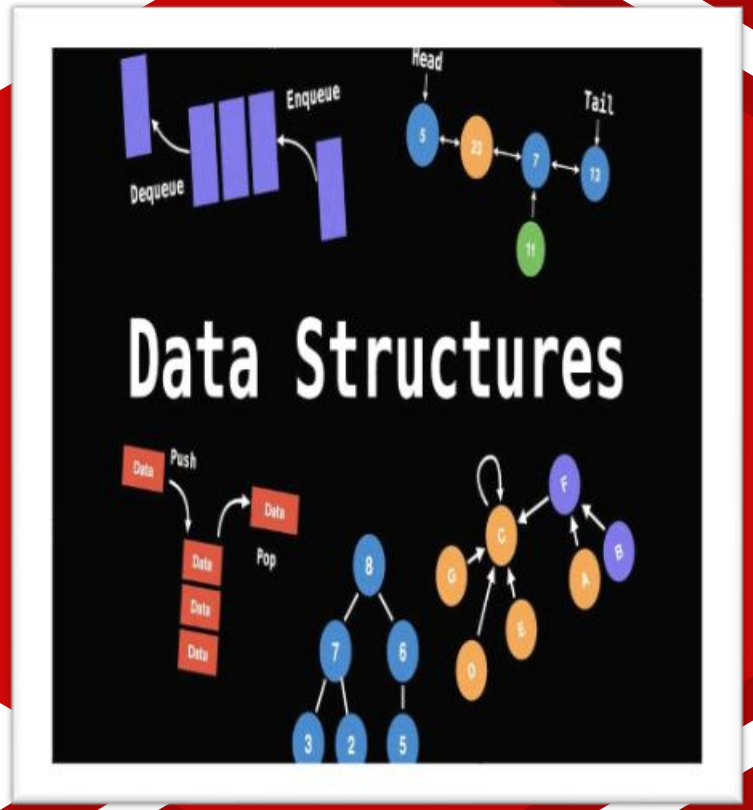
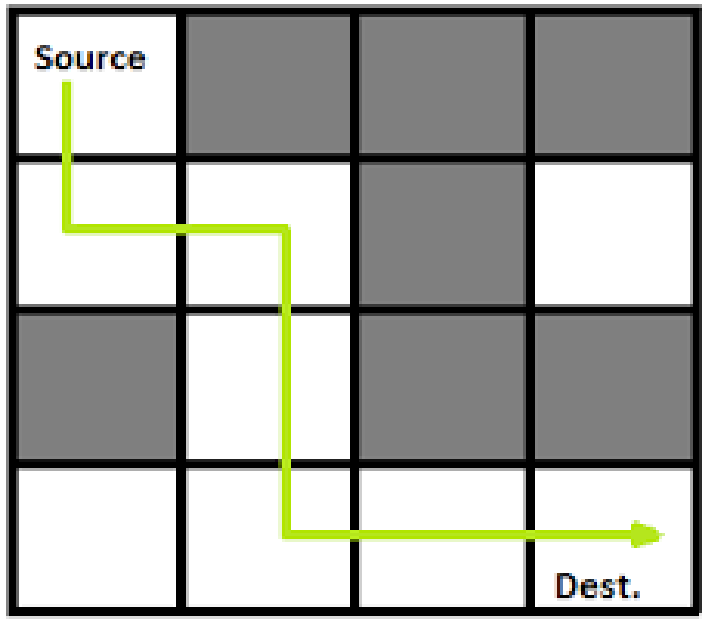
Algorithms and Data Structures

Backtracking

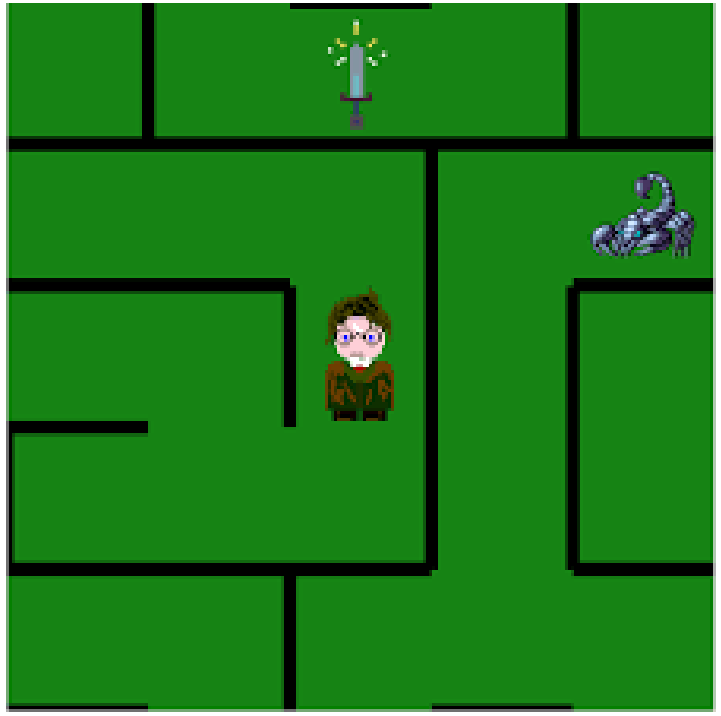
Session : Day 3

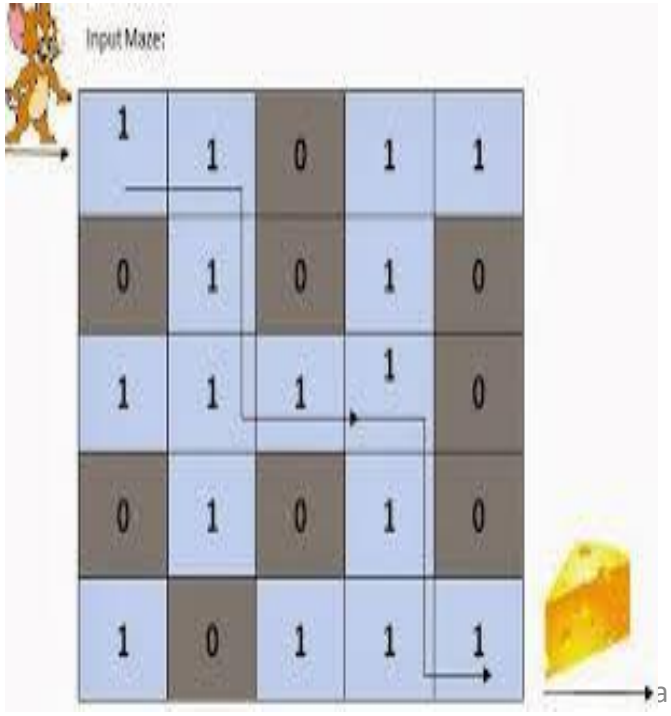
Dr Kiran Waghmare
CDAC Mumbai



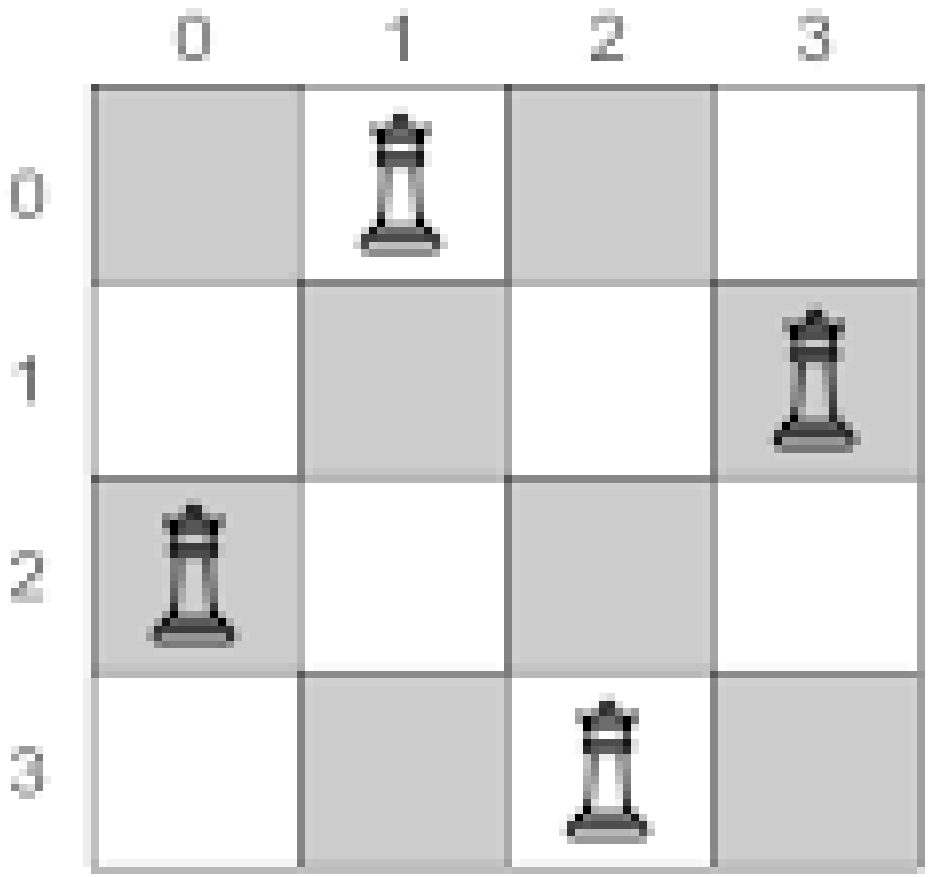

Source
Dest.


4
20
00:25.86

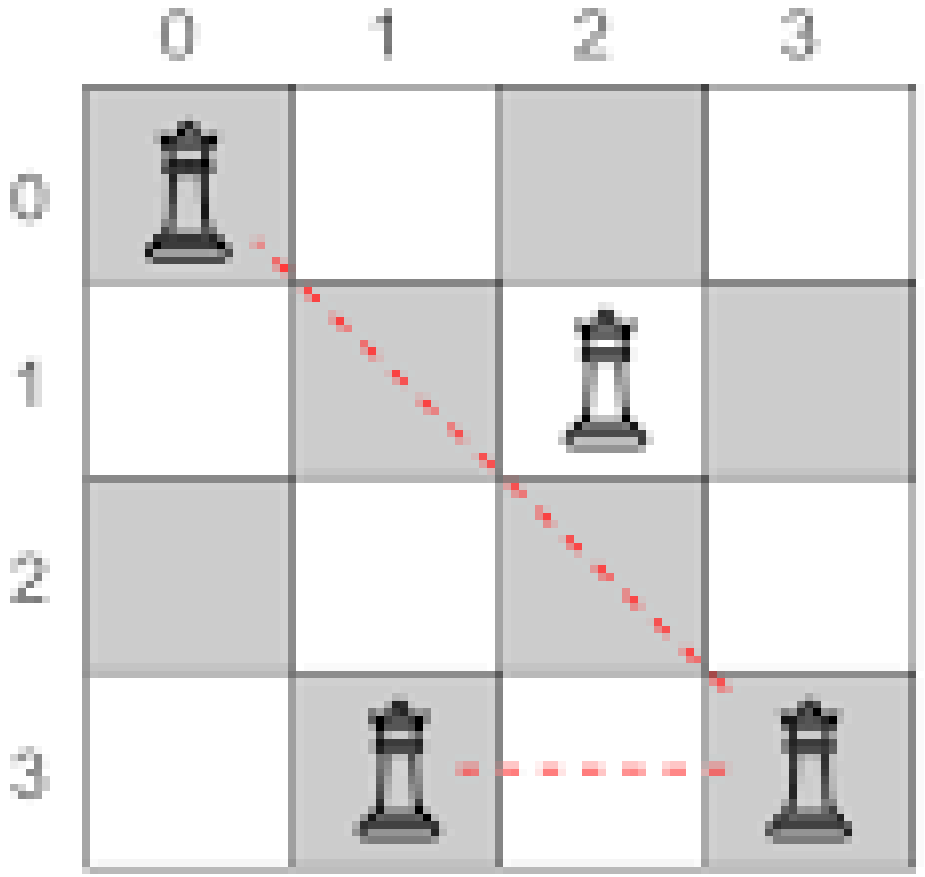



Input Maze:
1 1 0 1 1
0 1 0 1 0
1 1 1 1 0
0 1 0 1 0
1 0 1 1 1

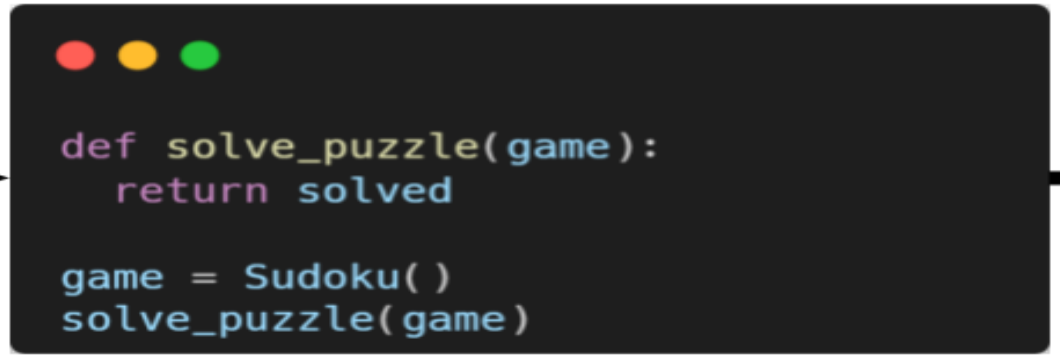
are



Valid queen positions



Invalid queen positions



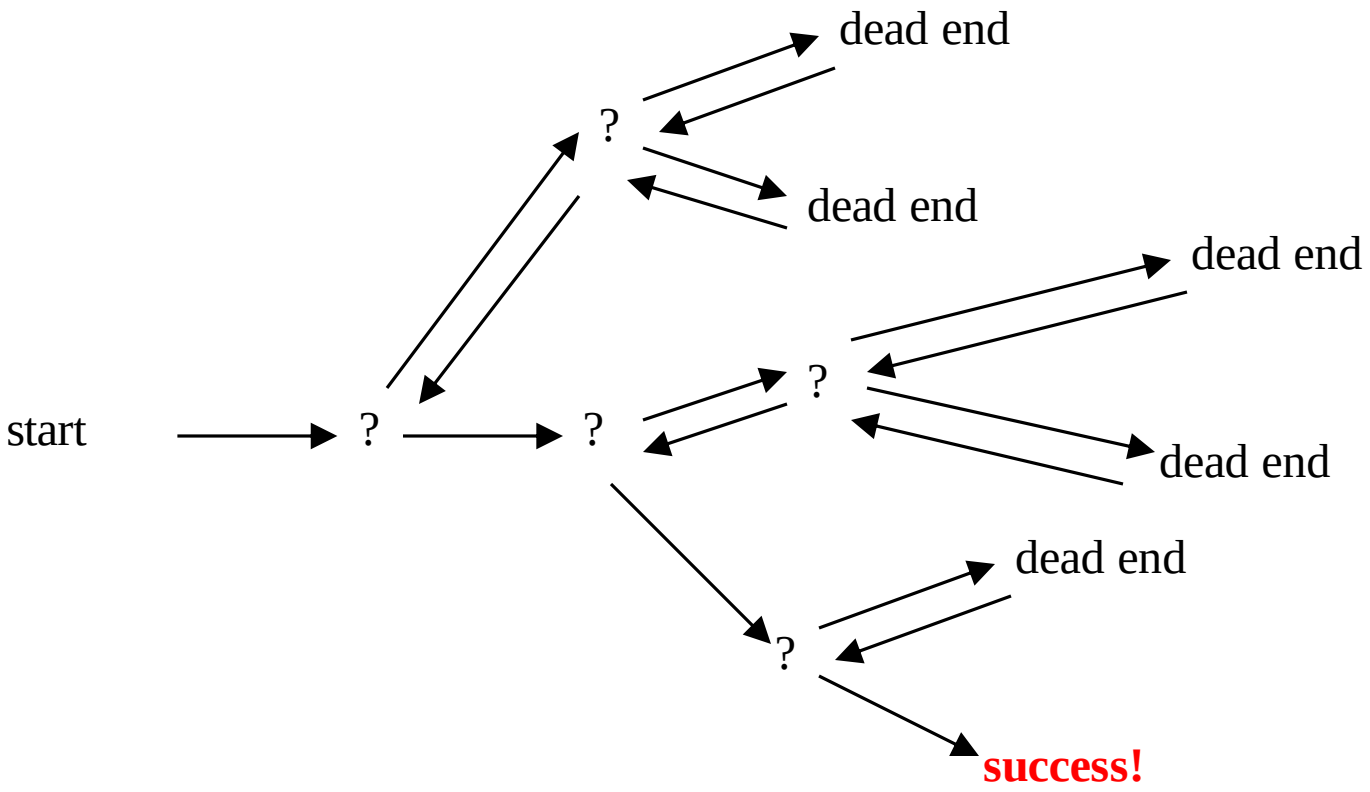
```
def solve_puzzle(game):
  return solved

game = Sudoku()
solve_puzzle(game)
```

		3			2	6	1	4
		2	6	4	1			8
	1	6		3	5		7	
			3				9	7
6	5				7		3	1
	3	7		5	4			
	7	9			6	2		
			5	8	3	7		
8	4			2			6	

5	8	3	9	7	2	6	1	4
7	9	2	6	4	1	3	5	8
4	1	6	8	3	5	9	7	2
1	2	4	3	6	8	5	9	7
6	5	8	2	9	7	4	3	1
9	3	7	1	5	4	8	2	6
3	7	9	4	1	6	2	8	5
2	6	1	5	8	3	7	4	9
8	4	5	7	2	9	1	6	3

Backtracking (animation)

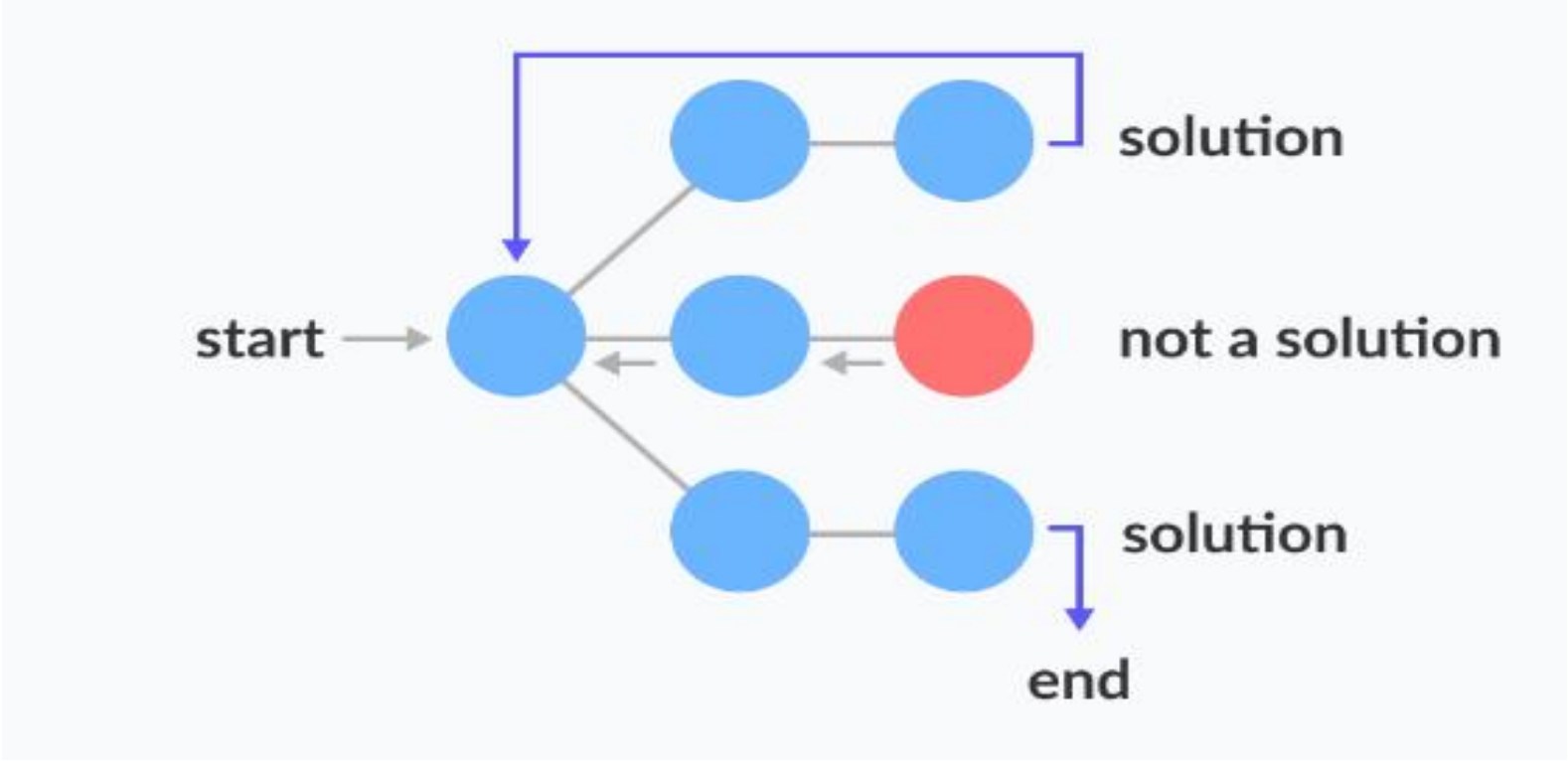


Backtracking

- Backtracking is a **problem-solving technique.**
- It involves systematically **exploring different paths** to find a solution.
- When faced with **multiple choices, backtracking tries each option.**
- It backtracks when it reaches **a dead end**.
- It's akin to navigating through a complex maze.
- **Wrong turns lead to retracing steps until the correct path** is found.
- Backtracking **enables the exploration of various possibilities.**
- It's a **powerful tool for tackling** challenging problems.

State Space Tree

- A space state tree is a tree representing **all the possible states (solution or nonsolution) of the problem** from the root as an initial state to the leaf as a terminal state.



- **Start**: Begin with an initial solution candidate or state.
- **Explore**: Examine all possible next steps or choices from the current state.
- **Constraint check**: Verify whether the current solution candidate satisfies the problem constraints or conditions.
- **Recursion**: If the current candidate satisfies the constraints, recursively explore further by making a choice and moving to the next state.
- **Backtrack**: If the current candidate does not satisfy the constraints, backtrack to the previous state and try a different choice or explore a different path.
- **Repeat**: Repeat steps 2-5 until a valid solution is found or all possible candidates have been explored.

- **Backtracking Algorithm**

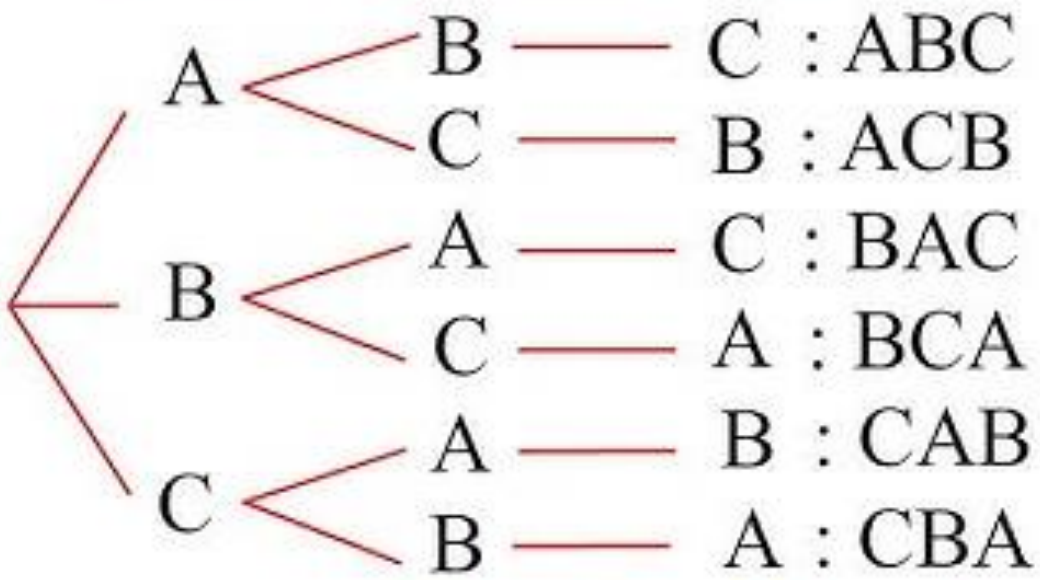
```
Backtrack(x)
    if x is not a solution
        return false
    if x is a new solution
        add to list of solutions
    backtrack(expand x)
```

- Backtracking **uses recursive algorithms** to explore potential solutions.
- **Each recursive call makes a choice and explores** further.
- If a dead end is reached, the algorithm backtracks to the previous state.

- Backtracking is commonly **used in problem-solving** scenarios.
- It's utilized in **constraint satisfaction problems, combinatorial optimization, sudoku solving, N-queens problem, graph traversal**, etc.

- Backtracking **efficiently searches through a large solution space**.
- It avoids unnecessary computations by pruning branches that cannot lead to a valid solution.

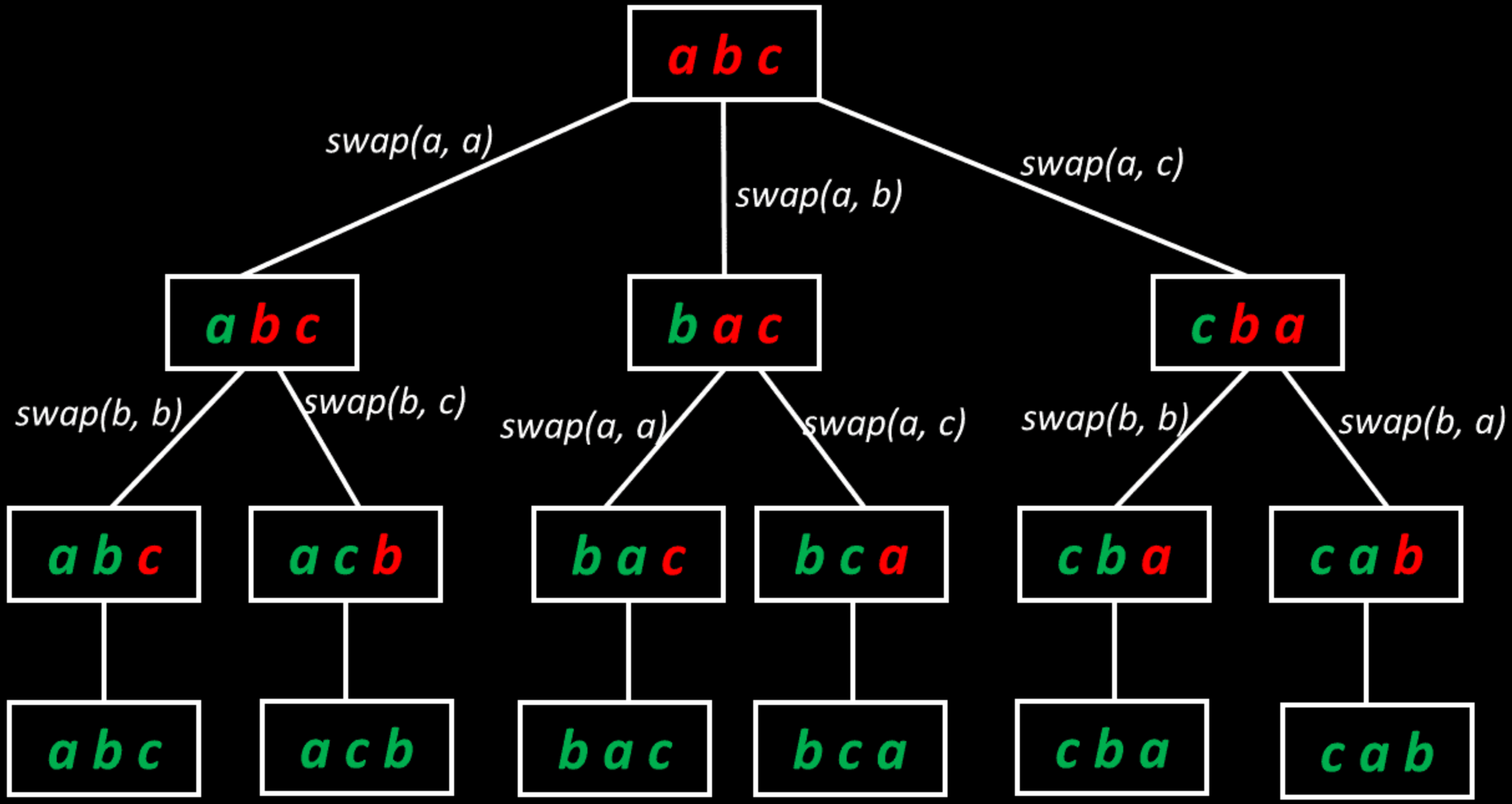
Permutations



Permutation of ABC

1st order 2nd order 3rd order

$3 \times 2 \times 1 = 6$ ways $= 3!$


a b c
swap(a, a)
swap(a, b)
swap(a, c)
a b c
b a c
c b a
swap(b, b)
swap(b, c)
swap(a, a)
swap(a, c)
swap(b, b)
swap(b, a)
a b c
a c b
b a c
b c a
c b a
c a b
a b c
a c b
b a c
b c a
c b a
c a b

```
748
749
750 display("ABC", "")
751 ├─ pick i=0, ch='A' → display("BC", "A")
752 │   ├─ i=0, ch='B' → display("C", "AB")
753 │   │   └─ i=0, ch='C' → display("", "ABC")   ==> print ABC   return
754 │   └─ i=1, ch='C' → display("B", "AC")
755 │       └─ i=0, ch='B' → display("", "ACB")   ==> print ACB   return
756 │   return to display("ABC","")
757 ├─ pick i=1, ch='B' → display("AC", "B")
758 │   ├─ i=0, ch='A' → display("C", "BA")
759 │   │   └─ i=0, ch='C' → display("", "BAC")   ==> print BAC   return
760 │   └─ i=1, ch='C' → display("A", "BC")
761 │       └─ i=0, ch='A' → display("", "BCA")   ==> print BCA   return
762 │   return to display("ABC","")
763 └─ pick i=2, ch='C' → display("AB", "C")
764     ├─ i=0, ch='A' → display("B", "CA")
765     │   └─ i=0, ch='B' → display("", "CAB")   ==> print CAB   return
766     └─ i=1, ch='B' → display("A", "CB")
767         └─ i=0, ch='A' → display("", "CBA")   ==> print CBA   return
768     return to display("ABC","")
769
770 Done. All calls return (void).
771
```